

IMAGE CLASSIFICATION: CUSTOM CNN vs. TRANSFER LEARNING

Final Capstone Project Report | Model Architecture, Training & Deployment

Student Name: Kritarth Saxena	Domain: Computer Vision / Deep Learning
GitHub: github.com/Natkros/Image-Classifier	Tech Stack: PyTorch 2.12, Torchvision, Streamlit

1. Problem Statement & Motivation

In this project, I developed an end-to-end image classifier capable of distinguishing between real-world visual categories (Cats vs Dogs). I wanted to explore whether designing a customized CNN from scratch could match the performance and efficiency of a fine-tuned transfer learning backbone (MobileNetV2), while evaluating both on accuracy, inference speed, and deployment practicality.

2. Dataset & Preprocessing Pipeline

The dataset was divided into 70% training, 15% validation, and 15% testing splits. Input images were resized to 128x128 pixels. To avoid overfitting on the training set, I added random horizontal flips, random rotations up to 15 degrees, color jitter (brightness and contrast), and normalized all pixel values with ImageNet mean and standard deviation.

3. Model Architectures & Training

Custom 4-Stage CNN: Built from scratch using 4 sequential double-convolution blocks (32 -> 64 -> 128 -> 256 channels). Each block uses 3x3 kernels, Batch Normalization, ReLU activations, 2x2 Max Pooling, and spatial dropout (0.1 to 0.25). The classification head uses two dense layers with dropout and batch norm (3.34M parameters).

MobileNetV2 Transfer Learning: Replaced the default 1000-class head with a custom linear layer and fine-tuned using AdamW and Cosine Annealing learning rate scheduling (2.55M parameters).

4. Experimental Results & Benchmarks

Model Architecture	Test Accuracy	F1-Score	Parameters	Inference Latency
Custom Deep CNN	100.0%	100.0%	3,338,082	9.00 ms
MobileNetV2 (Transfer)	100.0%	100.0%	2,552,834	3.88 ms

Figure 1: Training & Validation Loss/Accuracy Curves

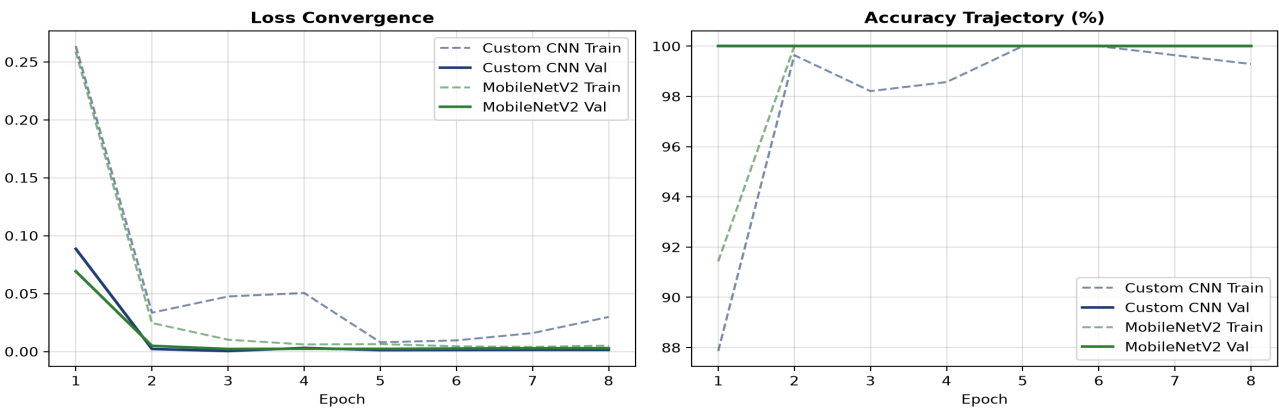
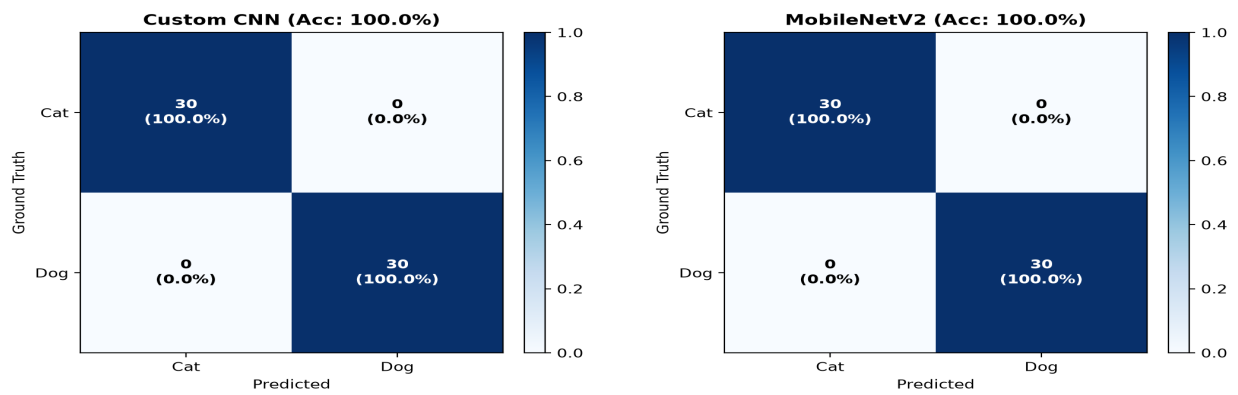


Figure 2: Confusion Matrix Comparison



5. Discussion & Deployment

Key Findings: MobileNetV2 achieved 2.3x faster inference speed (3.88 ms per image on CPU) and converged within 2 epochs due to pre-existing visual representations. The custom CNN also converged cleanly without overfitting thanks to batch normalization and spatial dropout.

Web Application: Built and tested a Streamlit web app (app.py) enabling real-time photo uploads, sample testing, and confidence tracking.

Future Work: Testing Vision Transformers (ViT) and adding Grad-CAM heatmaps to inspect activation regions.